TEACHING NOTES

Using a Sandboxed Ticketing Website to Teach Digital Trust and IT Audit

Case Files

This case includes the following supplemental materials:

- **Student Case Instructions:** student-case-instructions.pdf provides the activity flow, instructions for both implementations, and student deliverables.
- **Student Template:** worksheets/worksheet-version-1.docx is the standard student worksheet, including COBIT 2019 questions, website transaction discovery, automation observations, policy evaluation, and internal-control review.
- **Alternative Implementation Student Template:** worksheets/worksheet-version-2.docx is the shorter student worksheet for transaction discovery, process narrative, automation observations, internal-control review, and debrief questions.
- **Instructor Solutions:** instructor-solutions.pdf provides suggested answers for both worksheet versions.
- **Instructor PowerPoint:** instructor-slides.pptx supports classroom preparation, the automation demonstration, and the debrief.
- **GitHub Repository:** https://github.com/ais-2026-anonymous/ticketing-system provides the website source code, instructional videos, and additional classroom resources.

Instructors should distribute the Student Case Instructions and the assigned worksheet. The Teaching Notes and Instructor Solutions are for instructor use and should not be distributed before the exercise.

Implementation Notes

Both implementations use a trial event, one main ticket-drop event, an instructor-led automation demonstration, backend evidence review, and a class debrief. Instructors may assign individual or group work and may use a competitive or noncompetitive format. The standard worksheet provides more structured audit documentation. The alternative version uses the same system and evidence with fewer written responses.

For the standard implementation, students complete the four case requirements using Version 1. For the alternative implementation, students complete the five sections in Version 2. Instructors may add the COBIT framework questions or the six-policy evaluation from the case if they wish to extend the shorter version. Neither worksheet requires a separate audit memo unless the instructor assigns one.

The automation demonstration normally follows the main ticket drop. Instructors who include the optional AI-assisted coding extension may introduce and test instructor-approved code during the trial event. The same scope limitation applies to both versions: all testing takes place within the SeatGate sandbox.

Grading Guidance

Table 1 provides a suggested allocation for the standard worksheet. Instructors can select weights within the ranges that total 100 percent. For the shorter alternative version, allocate points to transaction discovery and the process narrative, automation observations, internal-control findings and recommendations, and the debrief responses. The emphasis should be on the quality of audit reasoning and supporting evidence rather than the number of tickets purchased.

Use the separate Instructor Solutions to guide discussion and grading. The answers are examples, not an exhaustive list. Students may identify additional weaknesses or propose alternative controls. Strong answers should connect observed system behavior to a control objective, explain the associated risk, and support the recommendation with audit evidence.

Exercise Notes

SeatGate is designed as a sandboxed IT audit case in which students act as junior IT auditors evaluating whether the system's stated control policies are actually enforced by system behavior and supported by audit evidence. The case connects AS 2201, COBIT and COSO to a live digital transaction process involving authentication, ticket inventory, purchase authorization, logging, exception monitoring, and backend evidence. The accompanying materials are provided to help instructors implement the exercise, guide student work, and debrief the case after the ticket-drop activity.

Instructors should clarify that the case uses a simplified setting to distinguish external IT audit work from broader operational review. In a financial-statement audit, the IT audit team generally focuses on controls that affect the reliability of financial reporting and system-generated evidence. The team may inspect backend code, database rules, or configuration settings to determine whether the implemented logic matches the documented control requirement. Internal audit, by contrast, is more likely to examine the broader operational aspects of the ticketing process, such as customer experience, platform fairness, business policy compliance, and ongoing operational monitoring.

Before the activity begins, the instructor should check the user accounts and ticket inventory in the admin panel. The maintained website is already set up. When preparing a new classroom run, the instructor can use the admin panel to reset the exercise data and release tickets. Students should use the trial event to learn the transaction flow and inspect the website.

After the trial event, students participate in one main ticket-drop event. The instructor then reveals a controlled automation script within the SeatGate sandbox. Instructors who wish to include an AI-assisted coding extension may introduce the script during the trial event so that students can understand, adapt, and test instructor-approved code before the main ticket drop. To generate student interest, instructors may award bonus credits to the student or group with the highest number of tickets. Instructors should clarify whether ticket purchases count toward the

course or case grade.

The instructor should release the main-event tickets at one time using the admin panel, with the quantity depending on the number of students. Releasing a controlled number of tickets creates excitement as students try to buy tickets in the shortest time. The instructor can adjust the available inventory to create a limited release for the class.

As an extension, students may use AI-assisted coding to understand how the website works, automate parts of the purchase process in the sandboxed environment, or test whether SeatGate's controls prevent or detect rapid purchase attempts. Instructors should clearly state that this activity is limited to the SeatGate sandbox. The purpose of the exercise is not to teach ticket scalping or circumvention of real ticketing systems. The purpose is to show how automated behavior can expose weak digital controls and why IT auditors must understand both front-end behavior and backend evidence.

After the main ticket drop and automation demonstration, the instructor should reveal selected backend reports. These may include transaction logs, failed purchase attempts, exception reports, user activity records, ticket ownership records, wallet balances, or other evidence relevant to the control review. The demonstration should help students compare manual purchasing behavior with automated purchasing behavior. Students should observe whether the system blocks rapid activity, logs suspicious behavior, enforces purchase limits, updates inventory correctly, and produces evidence that management could review.

Table 6 provides one possible implementation structure.

Student performance can be assessed using both technical and audit-based deliverables. The ticket-drop result provides evidence that students engaged with the website and understood how digital transactions can be affected by timing, automation, and control design. However, the main grading emphasis should be placed on the quality of students' audit reasoning. Instructors may choose to make the ticket-purchase outcome a small portion of the grade or treat it as participation only. This reduces the risk that students focus on "winning" the ticket drop rather than understanding the control implications.

The case may be assessed using two sources of evidence: the assigned version of the case worksheet and a post-case perception survey. The worksheet asks students to document their observations from the ticketing website, the ticket-purchase process, the automation demonstration, and the backend evidence provided by the instructor. These responses allow the instructor to assess whether students can connect technical observations, such as rapid purchasing, exposed page elements, backend validation logic, processing alternatives, purchase-limit behavior, and backend logs, to audit concepts such as authorization, transaction validity, monitoring, exception reporting, and server-side validation. The post-case perception survey asks whether the exercise improved students' understanding of IT audit, internal controls, automated purchasing risks, AI-assisted coding, and the importance of technical fluency for auditors. Table 1 provides a suggested grading allocation for the standard worksheet; instructors can adapt the allocation for the shorter alternative version.

The instructor debrief should emphasize that a transaction can be recorded by the system but still fail to satisfy the underlying control objective. Front-end restrictions are not sufficient if important controls are not enforced on the server side. A valid audit trail should include enough information to evaluate user identity, timing, authorization, inventory effects, and exception behavior. Automated purchasing risk is not only a technical issue; it is also an internal-control, monitoring, and governance issue. COSO provides the broad internal-control lens, while COBIT helps translate that lens into IT-control objectives such as access restriction, processing integrity, logging, exception monitoring, and management review.

The SeatGate environment is a sandboxed classroom system. Instructors should remind students that automation should not be used on real ticketing websites or external systems. The case should be framed as an IT audit and internal-control exercise, not as an exercise in ticket scalping. The appropriate learning objective is to understand how digital transactions are happening under the hood.

After the debrief, instructors may use the separate Instructor Solutions to guide discussion and grading for either worksheet version. The examples are not exhaustive; students may identify

additional weaknesses or propose alternative controls. Strong answers should connect the observed system behavior to a specific control objective, explain why the weakness matters, and recommend a control that would either prevent the issue or help management detect and review it.

Classroom User Roles and Embedded Audit Evidence

The SeatGate classroom environment uses several predefined user roles. Each student receives an anonymous username, such as `student1`, and a password from the instructor. These accounts allow students to participate in the trial and main ticket releases without placing real student names in the public repository.

The system also contains a deliberately created user called `fakebuyer`. This account represents an embedded control issue in the case. It appears in the system alongside the ordinary users and may hold tickets even though it does not correspond to an actual student participating in the exercise. The account is intended to prompt students to consider whether management adequately reviews the user population, identifies unauthorized or fictitious accounts, and reconciles ticket ownership with approved users. Students are not told the significance of this account before the exercise because identifying it forms part of the audit investigation.

The privileged admin account supports the classroom exercise. A user who signs in with the `admin` account can access the administrator panel shown in Figure 4. The panel displays the total number of users, tickets sold, revenue, trial and main ticket availability, ticket inventory, and the ticket holdings of all users. It also permits the administrator to reset the classroom database and release additional tickets. The panel therefore functions both as an operational interface and as a source of system-generated audit evidence.

A user who signs in with the `admin` account can also access the instructor demonstration panel shown in Figure 5. This panel contains an **Automated Buy** button and pseudocode describing the automated purchase sequence. The demonstration authenticates the instructor, retrieves the available ticket type, and sends a purchase request for five tickets in one transaction.

The "automated buy" panel demonstrates that restrictions presented through the ordinary

student interface may not be sufficient if equivalent restrictions are not enforced by the backend. After the automated purchase, students can compare the admin's ticket holdings, the number of tickets sold, and the remaining inventory in the administrator panel. This comparison allows students to examine whether the transaction was authorized, whether inventory was updated correctly, whether the unusual quantity was detected, and whether management received sufficient evidence to investigate the activity.

1 Instruction Manual

This case is accompanied by Student Case Instructions, two versions of the student worksheet, Instructor Solutions for both versions, three instructional videos, and a public GitHub repository. Instructors can use the maintained website immediately or fork the repository and deploy a customized version. The setup steps below are needed only for instructors who choose to deploy their own version.

Instructors who are using the default setup can use the site at https://ais-2026-anonymous.github.io/ticketing-system/, which we will maintain for one year. Instructors can log in using the default admin username `admin` and password `adminpass2`. The default setup also includes 50 student accounts, with usernames `student1` through `student50`. Each student's password is the same as their username; for example, the password for `student1` is `student1`. Instructors can assign these accounts to students before class and begin the activity without setting up their own GitHub repository or Render deployment.

Instructional Videos

The three instructional videos are linked from the repository README:

- **Setup Instructional Video:** optional GitHub Pages and Render deployment.
- **Classroom Briefing Video:** introduction to the case and debrief of the control failures.

- **Observing Network Flow Video:** inspection of the requests sent between the browser and the backend.

Case Materials

The two worksheet versions and Instructor Solutions are provided as supplemental materials, as listed above. Instructors can select the worksheet that best fits their class. The repository's `materials` directory contains the student instructions and both worksheet versions. Its `instructor` subdirectory contains the Teaching Notes, Instructor Solutions, and instructor PowerPoint:

https://github.com/ais-2026-anonymous/ticketing-system/tree/main/materials

The complete public GitHub repository, including the source code and supporting materials, is available at:

https://github.com/ais-2026-anonymous/ticketing-system

1.1 Using the Maintained Website

We will keep the SeatGate website running for instructors and reviewers. To begin, open the website link provided in the GitHub repository above. Log in to SeatGate with the username `admin` and password `adminpass2`. The admin account provides access to the administrator panel and the **Automated Buy** demonstration. Readers can log in immediately without creating a GitHub account, forking the repository, or setting up Render.

The maintained website also includes 50 student accounts, `student1` through `student50`. Each account initially uses its username as its password. Instructors can assign these accounts to students and distribute the Student Case Instructions with either worksheet version.

Before the activity, check the user accounts and ticket inventory in the admin panel. Use the panel to prepare the trial and main ticket releases for the classroom run. Instructors using the

maintained website can proceed directly to Section 1.10 to check the website and Section 1.11 for the automation demonstration.

The current repository includes a configurable backend ticket limit and rate limiting. Record the configured limit before class and compare it with the case's stated one-ticket-per-student-per-event policy. Use an ordinary student account to test this policy; the privileged admin account is exempt from the student limit. Review rate-limit audit entries when evaluating logging and exception reporting.

Optional Customized Deployment

Instructors who need to edit the source code, customize accounts, or run a separate classroom instance can fork the repository and deploy their own version using GitHub Pages and Render. The following steps summarize the setup process demonstrated in the Setup Instructional Video and apply only to this option.

1.2 GitHub Setup

For a customized deployment, instructors should first create a GitHub account if they do not already have one.

After signing in, go to:

https://github.com/ais-2026-anonymous/ticketing-system

Click **Fork** to copy the repository into your own GitHub account. Rename the forked repository to something you can remember. The instructional video uses:

`ticketing-bot`

Record the repository name because it must be entered in the frontend configuration later.

1.3 Render Database Setup

Go to:

https://render.com

Create an account. Then, create a new Render project. Within the project, start a new service and select "PostgreSQL database". This will be the database that stores the users, ticket inventory, and ticket-purchase activity.

Use any database name, although `ticketing-bot` is recommended for consistency. Keep everything else blank. Select the Free plan.

The free database may expire or wind down after approximately one month. Instructors should therefore avoid creating it too far in advance of the class. If it expires, a replacement database can be created.

1.4 Render Web Service Setup

While the database is being created, add a new Web Service to the same Render project. Connect the Web Service to the GitHub repository forked earlier.

Use any service name, although `ticketing-bot` is recommended. When creating the Render Web Service, enter the following settings:

Root directory: `server`

Build command: `npm install`

Start command: `npm start`

Instance type: Free

Click **Deploy** to create the Web Service.

When the PostgreSQL database is ready, open the database page and copy its **Internal Database URL**. Return to the Web Service, open the **Environment** settings, and create the following environment variable:

```
DATABASE_URL
```

Paste the Internal Database URL as its value and save the changes.

1.5 Frontend and Server Configuration

Return to the forked GitHub repository and open:

`vite.config.ts`

Edit the `base` value so that it matches the name of the forked repository. For example, if the repository is named `ticketing-bot`, use `/ticketing-bot/` as the base path. This is the name that you previously used after forking. Commit the change.

Next, return to the Render Web Service and go to **Settings**. Under **Custom Domains**, find the line "Your service is reachable at ..." and copy the address. Do not include punctuation that is not part of the address.

With this copied address, go to GitHub and open:

`src/.env.production`

Replace the existing backend address with the public Render Web Service address and commit the change.

Next, open:

`server/index.js`

Locate the `origin` setting in the `cors` configuration. Change the GitHub Pages origin from https://ais-2026-anonymous.github.io to the origin for your own GitHub username or organization. For example, replace `YOUR-USERNAME` below with the account that owns your fork. The origin does not include the repository path. Commit the change.

```
origin: 'https://YOUR-USERNAME.github.io',
```

1.6 Create the Database Secret in GitHub

Next, return to the forked GitHub repository and open the repository's **Settings** tab. In the left-hand menu, select **Secrets and variables**, then select **Actions**.

Under **Repository secrets**, click **New repository secret**.

For the secret name, enter `DATABASE_URL`.

For the secret value, paste the **External Database URL** from the Render PostgreSQL database. This is different from the **Internal Database URL** and is found in the settings of the PostgreSQL database.

Click **Add secret** to save it.

Make sure the secret name is entered exactly as `DATABASE_URL`. This secret allows the GitHub Actions workflow to connect securely to the PostgreSQL database without placing the database connection string directly in the public repository.

1.7 GitHub Pages Deployment

Go to **Settings** in GitHub, click on **Pages** in the left panel, and under **Build and deployment**, change the source to **GitHub Actions**.

Next, go to the **Actions** tab in GitHub. Run the GitHub Pages deployment workflow to publish the frontend. For a new blank database, run the **Initialize Database** workflow once, using the `DATABASE_URL` secret created above and entering `INITIALIZE` in its confirmation field. After initialization, use the admin panel to reset the classroom data when needed. The repository README provides the workflow details.

1.8 Keeping the Render Service Active

Render's free Web Service may wind down when it is inactive. The repository includes a GitHub Actions workflow that periodically contacts the backend.

In the repository, open:

```
.github/workflows/keep-alive.yaml
```

Replace the existing service domain with the public domain copied from Render. Preserve the health-check path:

`/api/health`

Commit the change. The workflow will periodically contact the health endpoint to reduce delays caused by the free service winding down.

1.9 Customizing Classroom Accounts

The maintained website already includes the admin account and 50 student accounts. To change or add accounts in your own deployment, open the following file in your forked GitHub repository:

`server/users.js`

The repository includes 50 default student accounts. Each account initially uses its username as its password and should be reviewed before the class.

Do not replace the usernames with real student names, because the repository may be publicly visible. Accounts should instead use anonymous identifiers such as:

`student01`, `student02`, `student03`, ...

Instructors may use an AI tool to generate unique passwords for the anonymous usernames. However, they should not submit the class roster or other identifiable student information to a commercial AI service.

The generated usernames and passwords can be copied into a private spreadsheet and matched with the instructor's class list locally. The spreadsheet can then be shown or distributed securely during class.

The account list should retain the following special accounts:

- one administrator account;
- one fake-buyer account used as part of the internal-control discussion; and
- the anonymous student accounts.

After changing the account list, commit the changes and run the appropriate deployment and database-initialization steps again if a new blank database is being used.

1.10 Opening and Checking the Website

For the maintained website, use the website link in the case repository. For your own deployment, wait until GitHub Pages deployment finishes, then go to:

Settings → Pages

Open the published website address.

Log in with the username `admin` and the default password `adminpass2`, or the credentials configured for your own deployment, to confirm that the admin panel loads correctly. The panel should display the users, tickets sold, revenue, trial-ticket availability, main-ticket availability, ticket inventory, and user ticket holdings.

Figure 8 shows the administrator view. The dashboard summarizes the number of users, tickets sold, revenue, and the remaining trial and main ticket inventory. It also allows the administrator to reset the classroom database, release additional tickets, and review the ticket holdings of all users. During the debrief, these records provide evidence for reconciling purchases with inventory and identifying unusual holdings.

Before the classroom activity, the administrator may reset the database and release the required number of tickets through the admin panel.

1.11 Instructor Demonstration

Log in using the `admin` account and open the **Automated Buy Demonstration**. Select the ticket class, set the quantity to five, and click **Buy Tickets**. Earlier versions use an **Automated Buy** button, as shown in the case figure.

The demonstration submits an automated request for five tickets in one transaction. Compare its result with purchases made through an ordinary student account. An admin purchase alone does not establish that the student purchase limit has failed.

After submitting the purchase, return to the admin panel and confirm that:

- the instructor's ticket holdings increased;

- the number of tickets sold increased;
- the available inventory decreased; and
- the transaction is visible in the administrative evidence.

The following pseudocode illustrates the automated sequence for the trial event:

```
token = SIGN_IN("admin", "adminpass2")


ticket = GET("/api/events/1/tickets").tickets[0]


POST("/api/purchase",
    headers = {
        "Authorization": "Bearer " + token
    },
    body = {
        "eventId": 1,
        "items": [
            {
                "ticketTypeId": ticket.id,
                "quantity": 5
            }
        ]
    }
)
```

The instructor can explain that the browser sends requests to the backend when a user signs in, retrieves ticket information, and confirms a purchase. The same request sequence can be identified through the browser's developer tools and reproduced by an automated program.

1.12 Classroom Debrief

After the ticket activity and instructor demonstration, students should evaluate whether the stated controls were properly designed and whether they operated effectively.

The instructor can use the resources in the repository's `materials` directory. Continue showing the second part of the Classroom Briefing Video to debrief the control failures and use the Observing Network Flow Video to explain how the browser communicates with the backend.

The discussion should address:

- whether the one-ticket purchasing restriction was enforced by the backend;
- whether only valid users could access the system;
- whether ticket inventory reconciled with completed purchases;
- whether automated purchases were prevented or merely recorded;
- whether exception logs were actively reviewed; and
- what management should change before using the system in production.

Students should support each conclusion with evidence such as screenshots, transaction results, inventory changes, ticket holdings, or log entries.

The main lesson is that frontend restrictions alone are not sufficient. Important controls must be enforced by the backend and supported by authorization rules, transaction limits, logging, exception reporting, and active monitoring.

FIGURES

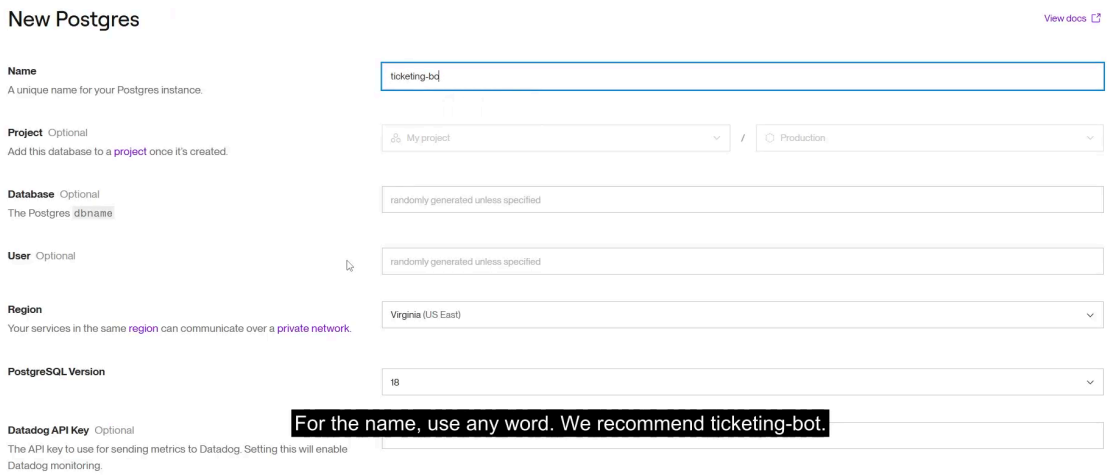


Figure 1: Name the database and keep all else blank

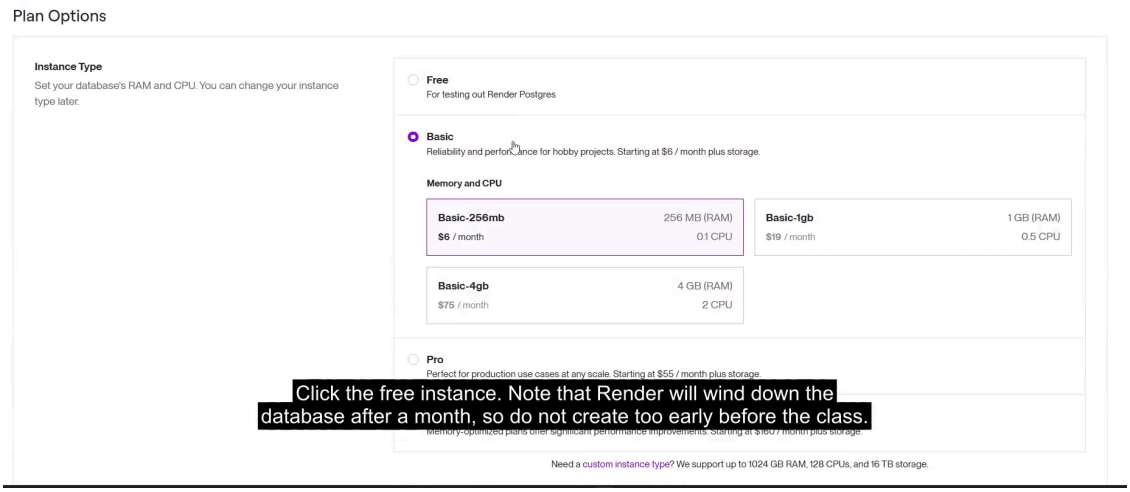


Figure 2: Select the free plan

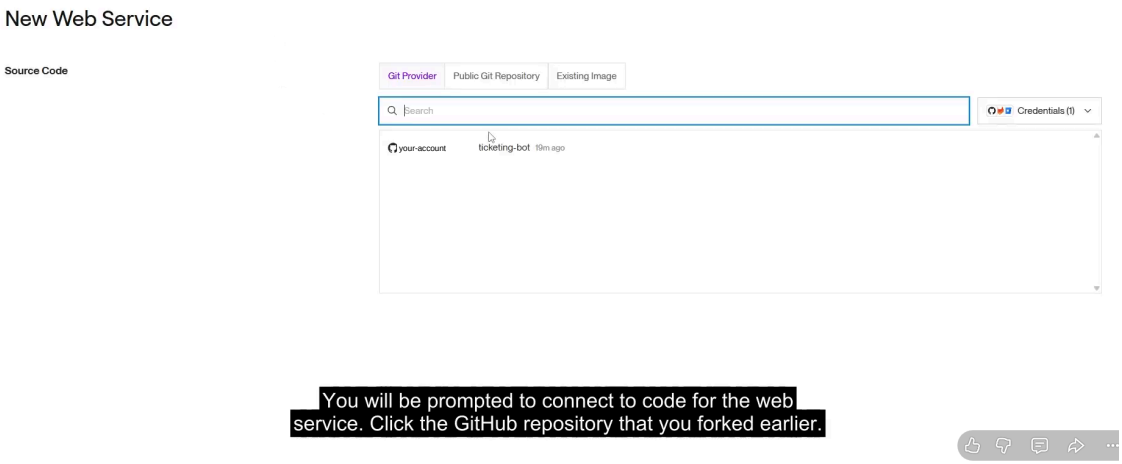


Figure 3: Select the repository forked into your GitHub account

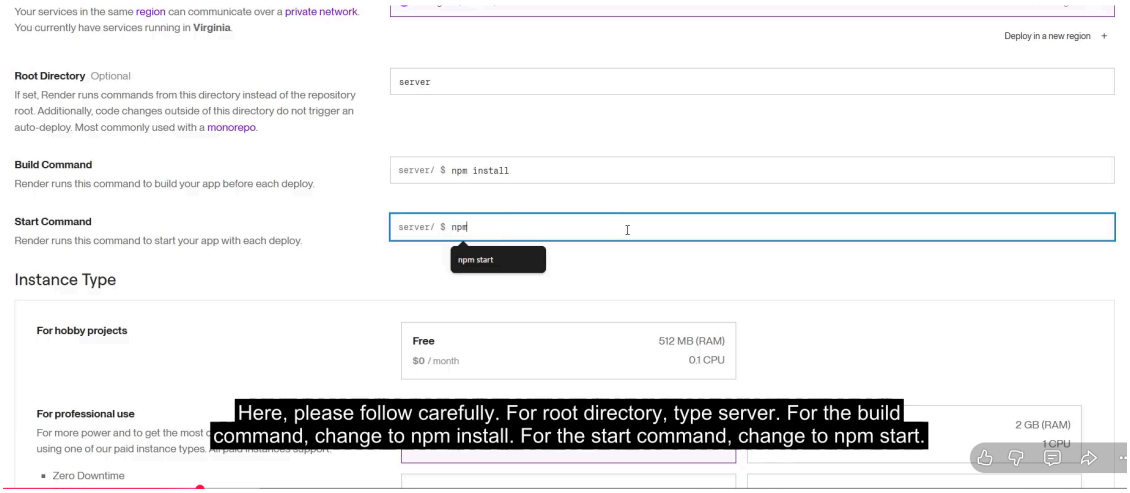


Figure 4: Follow these settings when creating the Render web service

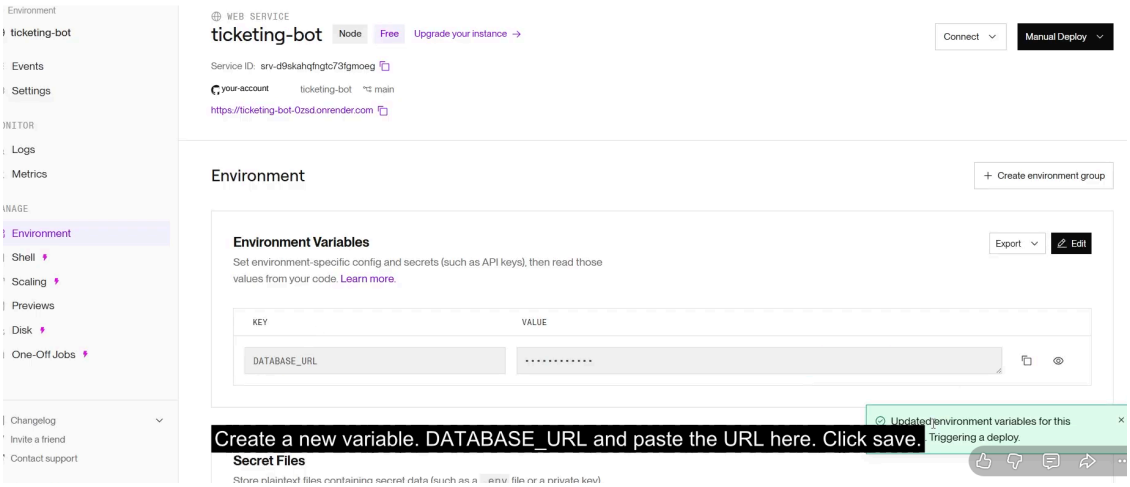


Figure 5: Create a new environment variable called `DATABASE_URL` and paste the "Internal Database URL" copied from the database information.

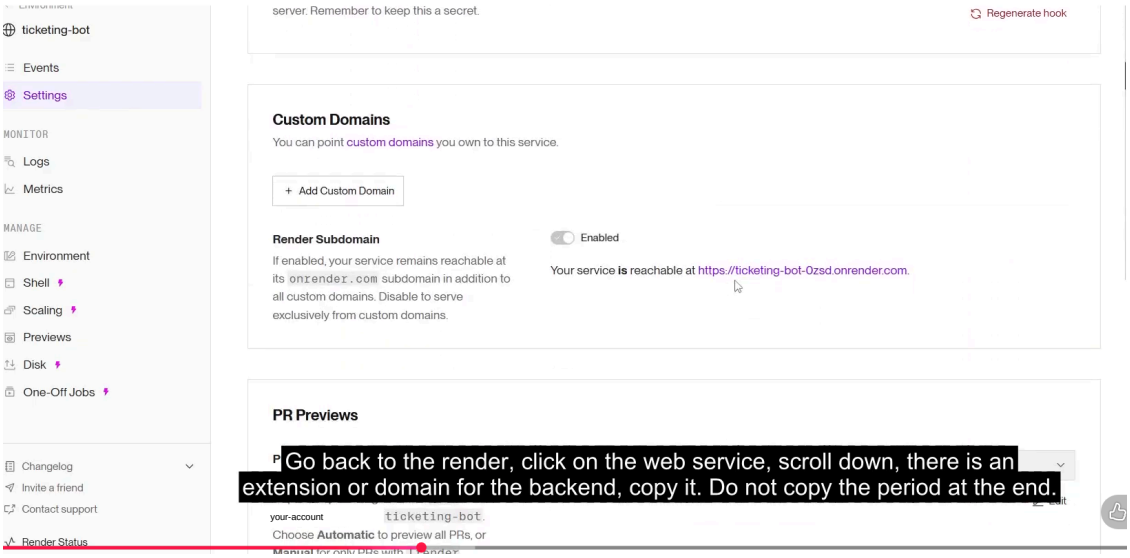


Figure 6: Copy the public address from the Render Web Service settings

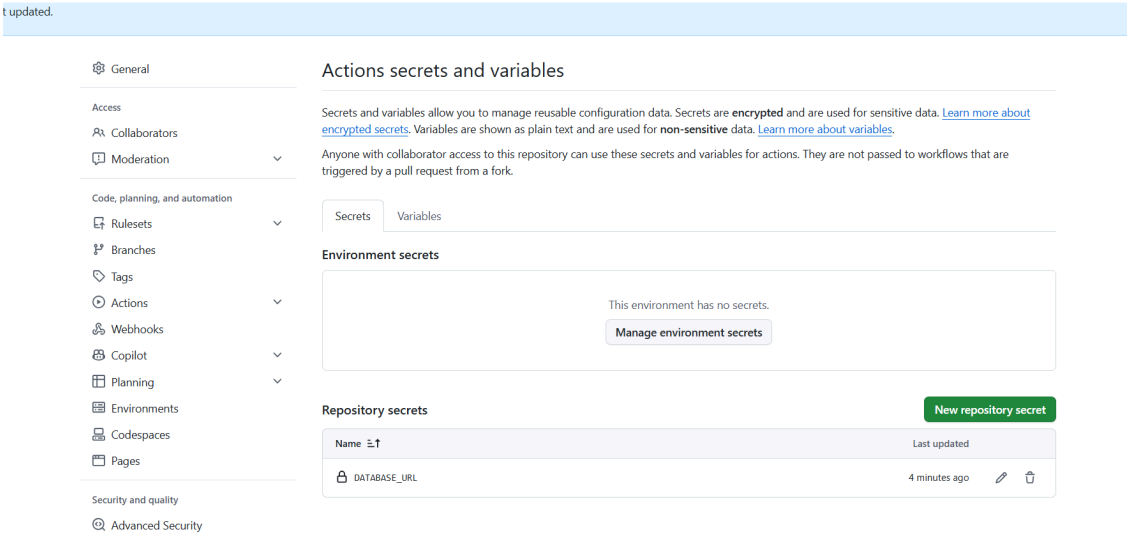


Figure 7: Create the `DATABASE_URL` repository secret under the GitHub settings, secrets and variables, actions

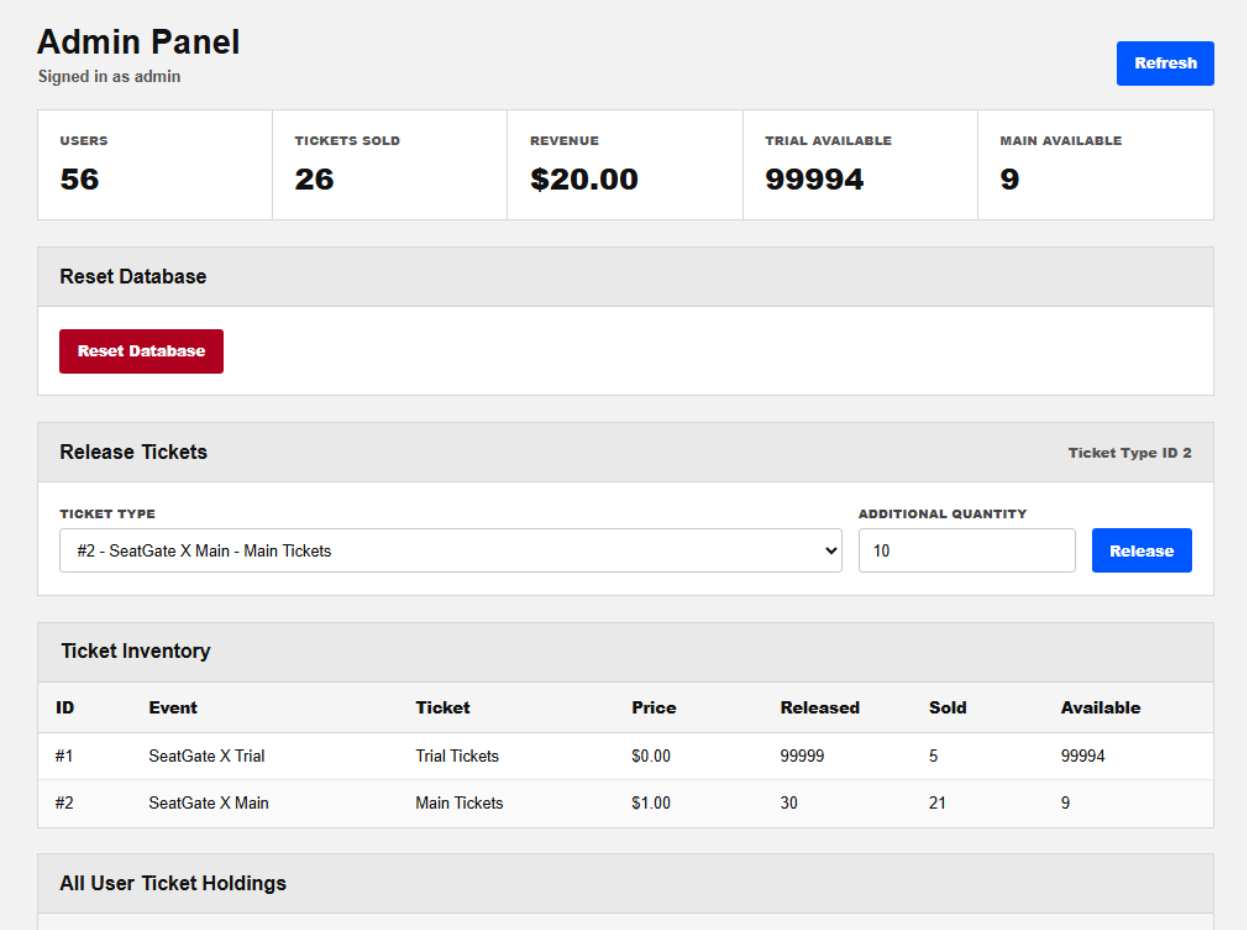


Figure 8: SeatGate administrator panel used to manage ticket releases and review user holdings

TABLES

Table 1: Suggested Grading Allocation

Component	**Suggested Weight**
Participation in ticket drops and sandbox activity	0–10%
COBIT 2019 Framework Questions	10%
Website Transaction Discovery Worksheet	20–30%
Policy Evaluation	20–30%
Internal-Control Review and COBIT 2019 Mapping	30%

Table 2: SeatGate Classroom Account Roles

Account	Role	Classroom Purpose
`admin`	Administrator	Resets the database, releases tickets, and reviews evidence
`fakebuyer`	Simulated user	Creates an intentional internal-control issue
`student1` through `student50`	Students	Participate in the trial and main ticket events